1) A) ArrayList extends one class and implements 6 interfaces:**Extends:** AbstractList<E>**Implements:** List<E>, RandomAccess, Cloneable, Serializable, Iterable<E>, Collection<E>

B) **Part B:** The Employee class does not override equals(). Without it, Java uses the default reference based equality from Object, so two Employee objects with the same name and salary are never considered equal. This means inList() always returns false and no duplicates are ever removed.

Part C: Same root cause as B Employee does not override equals() and hashCode(). Since HashMap relies on hashCode() to find the bucket and equals() to confirm a key match, tracker.containsKey(e) always returns false and every employee gets added, so no duplicates are removed.

Part D: Unlike B and C, Employee does override equals() and hashCode(), but the visited field is incorrectly included in the comparison. When a duplicate is found, setVisited(true) changes the stored employee's state. Later, listsAreEqual() compares those employees (with visited=true) against the expected list (with visited=false), and since visited is part of equals(), they don't match causing the test to fail even though the correct employees were kept.

When D is a class: Java classes can only extend one class, so D cannot directly extend both B and C simultaneously. The diamond problem doesn't arise for classes because Java simply forbids multiple class inheritance. D must pick one parent class and implement the other as an interface.

ii. When D is an interface: This is where it gets interesting. If B and C are interfaces with default method implementations /Java 8's behavioral inheritance, and D is also an interface that extends both, Java requires D to **explicitly override** method() to resolve the ambiguity. If D doesn't override it, the compiler throws an error. If D is a class instead, same rule applies it must override method() itself, or explicitly call one parent's version using B.super.method() or C.super.method()

2)

```
public sealed interface Expr permits Constant, Add, Multiply {

    int eval();
```

```
}
public record Constant(int value) implements Expr {
    public int eval() {
        return value;
    }
}
public record Add(Expr left, Expr right) implements Expr {
    public int eval() {
        return left.eval() + right.eval();
    }
}
public record Multiply(Expr left, Expr right) implements Expr {
    public int eval() {
        return left.eval() * right.eval();
    }
}
public class Main {
    public static void main(String[] args) {
        // (2 + 3) * 4
        Expr expr = new Multiply(
            new Add(new Constant(2), new Constant(3)),
            new Constant(4)
        );
        System.out.println(eval(expr)); // prints 20
    }

    public static int eval(Expr expr) {
```

```
        return switch (expr) {
            case Constant c  -> c.value();
            case Add a       -> eval(a.left()) + eval(a.right());
            case Multiply m  -> eval(m.left()) * eval(m.right());
        };
    }
}
```